

QUESTION-9

Stacked Using Linked List

main.c	Output
<pre>1 #include <stdio.h> 2 #include <stdlib.h> 3 struct Node { 4 int data; 5 struct Node *next; 6 }; 7 struct Node *top = NULL; 8 void push(int value) { 9 struct Node *newNode; 10 newNode = (struct Node *)malloc(sizeof 11 (struct Node)); 12 if (newNode == NULL) { 13 printf("Stack Overflow\n"); 14 return; 15 } 16 newNode->data = value; 17 newNode->next = top; 18 top = newNode; 19 printf("%d pushed into stack\n", value); 20 } 21 void pop() { 22 struct Node *temp; 23 if (top == NULL) { 24 printf("Stack Underflow\n"); 25 } 26 }</pre>	<pre>Original List: Linked List: 10 -> 20 -> 30 -> 40 -> 50 -> NULL After Deletion at Beginning: Linked List: 20 -> 30 -> 40 -> 50 -> NULL After Deletion at End: Linked List: 20 -> 30 -> 40 -> NULL After Deletion at Position 2: Linked List: 20 -> 40 -> NULL === Code Execution Successful ===</pre>

main.c	Output
<pre>26 temp = top; 27 printf("Deleted Element: %d\n", top->data 28); 29 top = top->next; 30 free(temp); 31 } 32 void peek() { 33 if (top == NULL) 34 printf("Stack is Empty\n"); 35 else 36 printf("Top Element: %d\n", top->data 37); 38 } 39 void display() { 40 struct Node *temp; 41 if (top == NULL) { 42 printf("Stack is Empty\n"); 43 return; 44 } 45 temp = top; 46 printf("Stack Elements:\n"); 47 while (temp != NULL) { 48 printf("%d\n", temp->data); 49 temp = temp->next; 50 } 51 }</pre>	<pre>Original List: Linked List: 10 -> 20 -> 30 -> 40 -> 50 -> NULL After Deletion at Beginning: Linked List: 20 -> 30 -> 40 -> 50 -> NULL After Deletion at End: Linked List: 20 -> 30 -> 40 -> NULL After Deletion at Position 2: Linked List: 20 -> 40 -> NULL === Code Execution Successful ===</pre>

main.c	Output
<pre>36 } 37 void display() { 38 struct Node *temp; 39 if (top == NULL) { 40 printf("Stack is Empty\n"); 41 return; 42 } 43 temp = top; 44 printf("Stack Elements:\n"); 45 while (temp != NULL) { 46 printf("%d\n", temp->data); 47 temp = temp->next; 48 } 49 } 50 int main() { 51 push(10); 52 push(20); 53 push(30); 54 display(); 55 peek(); 56 pop(); 57 display(); 58 return 0; 59 }</pre>	<pre>Original List: Linked List: 10 -> 20 -> 30 -> 40 -> 50 -> NULL After Deletion at Beginning: Linked List: 20 -> 30 -> 40 -> 50 -> NULL After Deletion at End: Linked List: 20 -> 30 -> 40 -> NULL After Deletion at Position 2: Linked List: 20 -> 40 -> NULL === Code Execution Successful ===</pre>